MODULE 18 · DOMAIN 2 - IMPLEMENT GENERATIVE AI AND AGENTIC SOLUTIONS

Agents II: tools and memory

Knowledge tools, Azure AI Search, OpenAPI, MCP and toolboxes, code interpreter, Content Understanding, and managed memory.

DOMAIN 2 - Generative AI and agents WEIGHT 30-35% MODULE 18 of 39

admin@sthithapragna.com

Every tool is a **door**. Choose the narrowest door that does the job, and give its keycard the **least access** it needs.

Agent tools, grouped by job

JOB	TOOLS
Find knowledge	File search, Azure AI Search, Foundry IQ, SharePoint, Fabric data agent
Find current public information	Web search, Grounding with Bing
Take actions	Function calling, OpenAPI, Azure Functions, MCP
Compute and analyse	Code interpreter
Operate interfaces (preview)	Browser automation, Computer Use
Remember people	Memory (preview)

ON THE EXAM

A tool needs support from both the model and the region. A2A, for calling other agents, is module 19.

Which knowledge tool?

TOOL	HOW IT WORKS	CHOOSE IT WHEN
File search	Upload files; the service chunks and embeds them	You're handing over a set of files
Azure AI Search	Queries an index you already run	An index exists, with its own schema and filters
Foundry IQ	Agentic retrieval over many sources, via MCP	Multiple sources, with permissions enforced
Web search	Searches the public web	Current public information is needed

ON THE EXAM

File search holds up to 10,000 files per vector store. Web search stays on a public endpoint even under network isolation.

Files you're handing over: **file search**. An index someone already runs: the **Azure AI Search tool**. Many sources with permissions: **Foundry IQ**.

Connecting an existing index

- Needs a **project connection** to the search service, and the **index name**.
- **query_type** defaults to **vector_semantic_hybrid**; **top_k** defaults to 5; a **filter** applies to every query.
- For citations, the index needs a retrievable **URL field**, and optionally a title.
- Keyless access: the project's managed identity needs **Search Index Data Contributor** and **Search Service Contributor**.
- Private search service: **Entra auth only**, keys unsupported, and the **standard** agent setup.

> THE SIGNAL
>
> 'Basic agent setup, private search index' doesn't work. The fix is standard setup with virtual network injection.

Four ways to let an agent act

TOOL	WHO RUNS THE CALL	CHOOSE IT WHEN
Function calling	Your application's code	Logic lives in your app, or needs local context
OpenAPI	Foundry calls the HTTP API from its spec	A REST API with a spec already exists
Azure Functions	A function triggered through a queue	Serverless logic; needs the standard setup
MCP	A remote MCP server	Tools shared across agents, or owned by another team

ON THE EXAM

OpenAPI needs no application code in the loop. Function calling always does.

What the specification must include

- An **OpenAPI 3.0 or 3.1** specification.
- Every operation needs an **operationId**, using only **letters, hyphens and underscores**.
- **Descriptive** operation names and summaries, so the model picks the right one.
- Request bodies as **application/json**, or application/json-patch+json.
- API keys: **one key scheme per OpenAPI tool**. Two schemes means two tools.

> WATCH FOR THIS
>
> The OpenAPI standard makes operationId optional. The Foundry tool doesn't.

Authenticating API calls

METHOD	HOW IT WORKS	USE IT FOR
Anonymous	No credentials sent	Public APIs, development and testing
API key	Key stored in a project connection	APIs that only accept keys
Managed identity	Entra token for the project's identity	Production APIs protected by Entra ID

THE TIEBREAKER

Managed identity: grant the project's identity a least-privilege role, and set the audience to the API's Application ID URI.

MCP servers, including Microsoft's own

- Connect a **remote MCP server**, and limit what's usable with **allowed_tools**.
- **require_approval** is on by default: keep it for anything consequential.
- Treat MCP tool descriptions and results as **untrusted input**, as module 10 showed.
- **Content Understanding, Language, Translator and Speech** are available as MCP servers in Foundry.
- So an agent can extract fields from a document by calling Content Understanding as a tool.

> ON THE EXAM
>
> Need document extraction inside an agent? Content Understanding through MCP, not a model reading the PDF on its own.

Configure once, reuse everywhere

- A **named, versioned** bundle of tool configurations, held in Foundry.
- Exposed as a single **MCP endpoint** that agents attach as one tool.
- Reused across **prompt agents, hosted agents** and frameworks like Agent Framework.
- Centralises **credentials, versioning and policy** for every agent that uses it.
- Microsoft now recommends adding Azure AI Search and OpenAPI tools through a toolbox.

> ON THE EXAM
>
> Ten agents each configuring the same search connection is the problem a toolbox solves.

Code interpreter

- Runs **Python in a sandbox**: calculations, data analysis, charts, file processing.
- Use it instead of trusting the model to do **arithmetic** or analyse a spreadsheet in its head.
- Charged **per session**, with a **5-minute minimum** and a 20-minute idle timeout.
- **Not available in every region**, whichever model you deploy.
- Under network isolation it runs over the Microsoft backbone, as module 07 showed.

> ON THE EXAM
>
> Precise numbers from uploaded data: code interpreter. A model 'estimating' totals is the distractor.

Three kinds of memory

TYPE	REMEMBERS	EXAMPLE
User profile	Durable preferences and facts about a person	'Allergic to dairy'; 'prefers email'
Chat summary	The gist of earlier conversations	'Last week: disputed a March invoice'
Procedural	How to do recurring work well	A routine that worked for this task type

ON THE EXAM

Session-only instructions, like 'just for this chat', belong in the conversation, not in long-term memory.

How managed memory works

- A **memory store** needs a **chat model** and an **embedding model** deployment.
- Attach the store to an agent, and memory search becomes one of its tools.
- **Scope** partitions memories. Set it **per user**, from the signed-in user's ID.
- Inject **profile memories** at the start of a conversation; search **contextual** ones each turn.
- Free during preview, apart from the chat and embedding model tokens it uses.

> ON THE EXAM
>
> To honour a deletion request, delete that user's memories by scope.

Creating a memory store

```
options = MemoryStoreDefaultOptions(
    user_profile_enabled=True,
    chat_summary_enabled=True,
    user_profile_details="Food preferences only. Avoid age, finances, "
                         "precise location and credentials.")
definition = MemoryStoreDefaultDefinition(
    chat_model="memory-chat",
    embedding_model="text-embedding-3-small",
    options=options)
store = project.beta.memory_stores.create(
    name="meal-planner-memory", definition=definition)
```

ON THE EXAM

user_profile_details steers what gets remembered. Scope, set when the agent calls memory, decides whose memories they are.

Memory done safely

- **Never share one scope across all users.** Microsoft recommends per-user access controls.
- If a scope must be shared, use user_profile_details to exclude personal information.
- **Minimise** what's stored, and redact sensitive data such as health or financial details.
- Tell users what's remembered, and let them **see and delete** it.
- Set a **TTL** so memories expire, rather than accumulating forever.

> THE SIGNAL
>
> 'The agent mentioned another customer's details' with memory enabled almost always means a shared scope.

Need to tool

THE AGENT NEEDS TO	USE
Answer from a folder of documents	File search
Query an index the data team maintains	Azure AI Search tool
Call an existing REST API securely	OpenAPI tool with managed identity
Run logic inside your own application	Function calling
Extract fields from uploaded invoices	Content Understanding, via MCP
Calculate totals from a spreadsheet	Code interpreter
Remember a customer's preferences	Memory, scoped per user

ON THE EXAM

Share the same tools across many agents through a toolbox.

> A manufacturer's data team runs an Azure AI Search index of two million product documents, with hybrid search and filters by product line.
>
> A new support agent must answer from those documents, with citations. The data team wants a single source of truth.

Which tool should the agent use?

A File search, with the documents uploaded into a new vector store.

B Web search, limited to the manufacturer's public product pages.

C The Azure AI Search tool, connected to the existing index.

D Code interpreter, reading an export of the documents each session.

◆ PAUSE AND DECIDE

Which tool should the agent use?

A File search, new vector store.

A second copy, and two million documents far exceed 10,000 files per vector store.

B Web search.

Public pages aren't the indexed documents, and the index has internal content.

C Azure AI Search tool.

Queries the existing index directly, with citations, keeping one source of truth.

D Code interpreter with an export.

Not a retrieval tool, and yet another copy of the data.

THE SIGNAL

'An index already exists' plus 'single source of truth' is the Azure AI Search tool.

> An agent needs to call an internal inventory API hosted on App Service. The API has an OpenAPI 3.1 specification and is protected by Microsoft Entra ID.
>
> The security team forbids storing API keys or secrets anywhere for this integration.

How should the team connect the API?

A. OpenAPI tool with anonymous access, restricted by IP address rules.

B. OpenAPI tool with managed identity, granted a least-privilege role.

C. OpenAPI tool with an API key written into the agent's instructions.

D. A function tool whose arguments include a bearer token from the model.

◆ PAUSE AND DECIDE

How should the team connect the API?

A Anonymous with IP rules.

Removes identity entirely, and the API requires Entra authentication.

B Managed identity, least privilege.

An Entra token for the project's identity, with no secret stored anywhere.

C API key in instructions.

A stored secret, in the least protected place possible.

D Token passed by the model.

Secrets in model-generated arguments can leak into traces and logs.

THE SIGNAL

'Protected by Entra ID' plus 'no stored secrets' is managed identity, every time.

> A clothing retailer's shopping agent uses managed memory. Every call uses the same fixed scope value, 'store-assistant'.
>
> A customer complains that the agent suggested sizes and a delivery address belonging to someone else.

What should the team change?

A Delete and recreate the whole memory store at midnight every day.

B Disable chat summary memory, and keep user profile memory as it is.

C Set a much shorter TTL so every memory expires within a few hours.

D Scope memory per user, and exclude addresses from profiles.

◆ PAUSE AND DECIDE

What should the team change?

A Recreate the store daily.

Destroys everyone's useful memory, and still mixes users within each day.

B Disable chat summaries.

Sizes and addresses are profile memories, which stay shared.

C Shorter TTL.

Shrinks the window, but customers still see each other's data inside it.

D Per-user scope, exclude addresses.

Partitions memory by person, and keeps sensitive details out entirely.

THE TIEBREAKER

One fixed scope for every user is one shared notebook. Scope is the keycard for memory.

> A team imports its logistics API's OpenAPI specification as an agent tool. Some operations work, but others never appear as usable.
>
> The failing operations either have no operationId, or have operationIds like 'get order/status'.

What should the team change?

A Give every operation an operationId of letters, hyphens and underscores.

B Convert the whole specification to the older Swagger 2.0 format instead.

C Switch the tool's authentication method from API key to anonymous.

D Raise top_k on the tool so that more operations are returned to the model.

◆ PAUSE AND DECIDE

What should the team change?

A Valid operationIds everywhere.

The tool requires one per operation, using only letters, hyphens and underscores.

B Convert to Swagger 2.0.

The tool supports OpenAPI 3.0 and 3.1, not the older format.

C Switch to anonymous.

Authentication affects calls, not whether operations are recognised.

D Raise top_k.

top_k belongs to the Azure AI Search tool, not OpenAPI.

THE SIGNAL

'Some operations work, others don't' points at the specification. Check operationIds first.

Tools and memory in five lines

- **Files:** file search. **Existing index:** AI Search tool. **Many sources:** Foundry IQ.
- **OpenAPI:** 3.0 or 3.1, valid operationIds, managed identity in production.
- **MCP and toolboxes** share tools; Content Understanding is available as an MCP server.
- **Code interpreter** for exact numbers; it bills per session.
- **Memory:** chat and embedding models, scoped per user, sensitive details excluded.

> ON THE EXAM
>
> Next, module 19: multi-agent orchestration with Agent Framework, A2A and hosted agents.